Collaborative Pixel Canvas with Rebalancing and Recoverable History

Academic Project Proposal | Team size: 2

1. Project Description

The project aims to develop a distributed, real-time collaborative pixel canvas in which multiple users can simultaneously draw on a shared 1000 × 1000 pixel grid. Users can select colours using HEX values, paint individual cells, undo eligible operations, and clear the canvas. Each user session receives a unique identifier to track operations and support user-aware undo.

The backend uses tile-based sharding, partitioning the canvas into 25 × 25 pixel tiles, resulting in 1,600 independently manageable tiles. These tiles are distributed across multiple shard processes. The system supports adding nodes and migrating tiles while drawing continues, ensuring concurrent writes are not lost and stale owners cannot modify migrated tiles.

To maintain consistency, the system uses deterministic update ordering, per-tile ownership epochs, operation identifiers, and duplicate-update detection. Persistent operation logs and coordinated snapshots enable recovery after process failures. The system also supports crash recovery during migration, restoration of canvas state, and replay of logged operations. Correctness is evaluated using canvas hashes and timelapse frames compared against reference history.

2. Goals

- **Real-time collaboration:** allow multiple users to draw on a shared canvas and receive updates through WebSocket connections.
- **Distributed sharding:** partition the canvas into tiles and distribute ownership across multiple backend processes.
- **Live migration and rebalancing:** transfer tile ownership between nodes without losing concurrent updates or unnecessarily interrupting drawing.
- **Deterministic consistency:** define a well-ordered sequence of committed updates and prevent stale owners from accepting writes using ownership epochs and version tracking.
- **Fault tolerance and recovery:** recover shard state after crashes using persistent logs, snapshots, and replay, including failures during migration.
- **Idempotent processing:** ensure retried or replayed operations are not applied more than once.
- **User-aware operations:** support up to ten undo attempts per user, without overwriting a newer update made by another user; record clear operations in the history.
- **Verification:** compare restored canvas hashes and timelapse frames against expected results from the operation history.

System Architecture

High-level view of client communication, shard ownership, persistence, and recovery.

Users' Web Browsers **HTML / CSS / JavaScript Canvas API**
↓ WebSocket + REST ↓
FastAPI Backend / Router **sessions • routing • ownership epochs**
↓ Tile routing and migration coordination ↓
Shard Processes: Shard A | Shard B | … | Shard N **Each stores its assigned tiles**
Snapshots + durable operation logs per shard
Recovery and replay • idempotency • hash/timelapse verification

Communication: WebSockets for live drawing updates; REST/HTTP for control actions. RCE may host multiple shard processes, but local execution remains the baseline.

3. Tools and Technologies

- **Backend and APIs:** Python and FastAPI.
- **Real-time communication:** WebSockets; REST over HTTP for control operations.
- **Frontend:** HTML, CSS, JavaScript, and the browser Canvas API.
- **Canvas representation:** NumPy arrays using RGB uint8 values.
- **Snapshot and image processing:** Pillow.
- **Persistence:** SQLite and durable operation logs.
- **Distributed execution:** separate Python shard processes, each with its own port and storage directory.
- **Testing and verification:** automated Python tests, hashing, and timelapse comparison.
- **Deployment:** publicly hosted frontend with a local or RCE-based backend.

4. Special System Requirements

Development can begin on a standard computer with Python, the required libraries, and a modern browser. The canvas is represented as a 1000 × 1000 × 3 RGB array, while each shard stores only its assigned tiles wherever practical.

Multiple shard processes run independently, using separate ports and persistent storage directories. The frontend will be hosted publicly so multiple users can access the same canvas. A secure networking arrangement, such as a suitable tunnel or hosted backend endpoint, will connect the public frontend to the backend.

The university Remote Computing Environment (RCE) is an optional resource for running multiple backend processes and demonstrating distributed execution; it is not mandatory for development.

Testing will include concurrent drawing, node addition and restart, tile migration, duplicate operation delivery, and controlled shard failures during migration. Persistent storage must remain available across process restarts. Replication across independent machines is outside the initial scope.

5. Expected Outcomes

The expected outcome is a functional distributed collaborative canvas demonstrating core distributed-systems concepts through an interactive application. Multiple users will draw simultaneously while the backend distributes work across shard processes and maintains consistent shared state.

The project will demonstrate live tile migration and rebalancing, ownership transfer, stale-write prevention, deterministic update ordering, snapshots, operation-log replay, and recovery following shard failures. Undo and clear operations will be integrated into the logged history.

The final demonstration will include adding a shard, migrating tiles while users continue drawing, crashing a shard during migration, restarting it, and verifying that the recovered canvas matches the expected state. Canvas hashes and timelapse frames will provide objective evidence of correctness.